

UAA12 – Manipuler les Bases de données en python et flask

1 TP1 : Manipuler les bases de données avec python

1.1 Test1 : Prise en main

1. Dans ton dossier uaa12_web, crée un `uaa12_python_flask_sqlite`, puis un sous-dossier `tp1_python_sqlite_test1`
2. Dans ce dossier, crée un fichier `test1-1_create.py` et suis le tuto `[1-AppPy]` Apprendre Python `p279-284` « Création de la base de données - Objets « connexion » et « curseur »

Remarque :

Que se passe-t-il si tu exécutes plusieurs fois ton programme ?

Pour pouvoir lancer indéfiniment ton script, utilise l'instruction `try except` :

```
try:
    cur.execute("CREATE TABLE membres(age INTEGER, nom TEXT, taille REAL)")
except:
    pass
```

Code test1-1_create.py

```
# Création de la bd - objets connexion et curseur p282-283
# Initialisation
#####
import sqlite3
fdb="bd_test.db" # fichier de la bd
                # SQLite mémorise toutes les tables d'une BD dans un seul fichier que l'on peut
sauvegarder où on veut
oconn=sqlite3.connect(fdb) #objet-connexion: interface entre le prog et la bd
                # connexion au fichier de BD (et création s'il n'existe pas encore)
cur=oconn.cursor() # curseur: mémoire tampon pour dialoguer avec la BD.
                # les données en cours de traitement et les opérations effectuées sont mémorisées
temporairement
                # avant leur transfert définitif ds la BD
                # Cela permet d'annuler si nécessaire des opérations inadéquates et de revenir en arrière
```

```

# Requetes SQL
#####

# Création d'une nouvelle table
try:
    cur.execute("CREATE TABLE membres(age INTEGER, nom TEXT, taille REAL)")

    # Entrer les enregistrements:
    cur.execute("INSERT INTO membres VALUES(21, 'Dupont', 1.83)")
    cur.execute("INSERT INTO membres VALUES(15, 'Blumar', 1.57)")
    cur.execute("INSERT INTO membres VALUES(18, 'Ozemir', 1.69)")
except:
    pass

# Syntaxe générale: cur.execute("requete sql")
# Attention, execute() crée les données dans la mémoire tampon cursor() mais pas ds la bd
# (ce qui permet de l'annuler si besoin)

# Sauvegarde et fermeture
#####
oconn.commit()    # Le transfert dans la base de données est fait par commit()
cur.close()       # fermeture du curseur
oconn.close       # fermeture de la connexion

```

3. Dans ce même dossier, crée un fichier **test1-2_connexion.py** et suis le tuto **[1-AppPy]** p284-286 « Connexion à une base de données existante »

Code test1-2_connexion.py

```

#####
# Connexion à une bd existante p284
# Initialisation
import sqlite3
oconn=sqlite3.connect("bd_test.db")
cur=oconn.cursor()

# Requetes SQL
print("Série de tuples")
cur.execute("SELECT * FROM membres") # sélection des enregistrements
for l in cur:                        # affichage des enregistrements
    print(l)                         # série de tuples

```

```

print("curseur ap un select *",cur)

print("Liste de tuples")
cur.execute("SELECT * FROM membres")
print(list(cur))

print("Liste de tuples avec fetchall()")
cur.execute("SELECT * FROM membres")
print(cur.fetchall())

print("Insertion de données par des var python")
cur.execute("INSERT INTO membres VALUES(18, 'Ricard', 1.75)")

data=[(17,"Durand",1.74),(22,"Berger",1.71),(20,"Weber",1.65)]
for tu in data:
    print("tu: ",tu)
    cur.execute("INSERT INTO membres VALUES(?,?,?)",tu) # insertion avec formatage de la
    requête par execute()

print("Modif un enregistrement")
cur.execute("UPDATE membres SET nom='Gerart' WHERE nom = 'Ricard'")
cur.execute("SELECT * FROM membres")
for l in cur:
    print(l)

print("Supprimer un enr")
cur.execute("DELETE FROM membres WHERE nom='Gerart'")
cur.execute("SELECT * FROM membres")
for l in cur:
    print(l)

# Sauvegarde et fermeture
oconn.commit()
cur.close()
oconn.close

```

4. Dans ce même dossier, crée un fichier **test1-3_exo16-1_create.py** et fais l'exercice **[1-AppPy] exercice 16.1 p286**

1.2 Test2 : Filmographie

1. Dans `uaa12_python_flask_sqlite`, crée un dossier `tp1_python_sqlite_test2`
2. Reprends les consignes et ton code sql du « TP3 - filmographie » de l'UAA7 BD

Tables des films

f_id	titre	realisateur	annee
1	Les évadés	Darabont	1994
2	Le parrain	Coppola	1972
3	Le parrain 2	Coppola	1974
4	L'odyssée de Pi	Ang Lee	2013
5	Chocolat	Hallstrom	2000
6	Scarface	De Palma	1983
7	Rango	Verbinski	2011

Tables des acteurs

a_id	nom
1	Johnny Deep
2	Al Pacino
3	Suraj Sharma
4	Brad Pitt
5	Edward Norton

Tables des filmographies

fg_fk_acteur_id	fg_fk_film_id	role	salaire
1	5	Roux	5000
1	7	Rango	10000
2	2	Michael Corleone	10000
2	3	Michael Corleone	20000
2	6	Tony Montana	15000
3	4	Pi	20000

3. Crée 2 fichiers `test2_create_insert.py` et `test2_setup.py`:
 - ✓ `test2_create_insert.py` contient les créations des tables et les insertions des données. Teste-le en essayant de lire les données par le terminal sql.
 - ✓ `test2_setup.py` contient la lecture du contenu de chaque table et des 3 jointes

Résultat dans le terminal

acteurs		filmographie		films	
a_id	nom	f_id	titre	realisateur	annee
1	Johnny Deep	5	Chocolat	Hallstrom	2000
1	Johnny Deep	7	Rango	Verbinski	2011
2	Al Pacino	2	Le parrain	Coppola	1972
2	Al Pacino	3	Le parrain 2	Coppola	1974
2	Al Pacino	6	Scarface	De Palma	1983
3	Suraj Sharma	4	L'odysséede Pi	Ang Lee	2013

1.3 Test3 : Injection SQL

- ✓ Reprends l'exercice `test1-3_exo16-1_create.py` de [1-AppPy] exercice 16.1 p286. Renomme-le `tp8_bd_3_exo16-1_injections_sql.py`.

On a vu que l'on pouvait écrire des requêtes paramétrées grâce au caractère `?`, et c'est vraiment ainsi qu'il faut procéder ! Mais à la place d'écrire :

```
req="INSERT INTO compositeurs (comp, a_naiss) VALUES (?, ?)"
```

on aurait pu être tenté d'écrire :

```
req = "INSERT INTO notes VALUES ('" + nom + "','" + aNais + "')" 
```

ou

```
req = f"INSERT INTO compositeurs VALUES ('{nom}',{aNais})"
```

Cette 2 dernières façons de faire, en construisant notre requête SQL comme une chaîne de caractère classique ou une chaîne formatée, introduit une faille de sécurité critique pour notre application, qui devient vulnérable à ce que l'on appelle une **injection sql**.

- ✓ Simplifie ton pour ne garder que la création de la table compositeurs (nom et année de naissance), la boucle d'insertion et la lecture des données de la table.

Code simplifié

```
try:
    req="CREATE TABLE compositeurs(comp TEXT, a_naiss INTEGER)"
    cur.execute(req)
except:
    pass

while True :
    nom = input("Nom du compositeur (<Enter> pour terminer) : ")
    if nom == '':
        break
    aNais = input("Année de naissance : ")
    req = "INSERT INTO notes VALUES ('" + nom + "','" + aNais + "')"
    print(req)
    cur.executescript(req)

cur.execute("SELECT * FROM compositeurs")
print(cur.fetchall())
```

- ✓ Teste-le avec les données suivantes et observe le contenu de la base de données :

Résultat

```
Nom du compositeur (<Enter> pour terminer) : roro', 2024); DROP TABLE compositeurs; --  
Année de naissance : 3000
```

Explication

La requête formulée est :

```
INSERT INTO notes VALUES ('roro', 2024); DROP TABLE compositeurs; --',3000)
```

Dans un 1^{er} temps, le couple ('roro', 2024) a été inséré.

Puis l'ordre a été donné de détruire la table

Le reste de code qui n'est pas correct est ignoré car - est le symbole du commentaire en sqlite.

Evidemment, ce code a été écrit pour être spécifiquement vulnérable à l'injection SQL. Il suffit de remplacer `cur.executescript(req)` par `cur.execute()` pour que le code reste fonctionnel et refuse l'injection (1 seule requête SQL peut être exécutée avec `execute(req)`)

2 TP2 : Manipuler les bases de données avec flask

⇒ Tuto TBD sur :

https://info-mounier.fr/terminale_nsi/base_de_donnees/index.php

2.1 Test1 : Prise en main - afficher une table dans le navigateur

Pour plus d'information, voir le tutoriel suivant « Étape 4 - Mise en place de la base de données »: [17] TP - Créer une appli web (css, sql, javascript)

<https://www.digitalocean.com/community/tutorials/how-to-make-a-web-application-using-flask-in-python-3-fr>

2.1.1 Mise en place de la base de données

Au cours de cette étape, vous allez créer une base de données pour stocker les données, c'est-à-dire les articles de blog de votre application. Vous allez également remplir la base de données avec quelques exemples d'entrées.

Vous utiliserez un fichier de base de données SQLite pour stocker vos données, car le module **sqlite3**, que nous utiliserons pour interagir avec la base de données, est disponible dans la bibliothèque standard Python. Pour plus d'informations sur SQLite, consultez ce tutoriel :

<https://www.digitalocean.com/community/tutorials/how-and-when-to-use-sqlite>

Tout d'abord, comme les données dans SQLite sont stockées dans des tableaux et des colonnes, et comme vos données consistent principalement en des articles de blog, vous devez d'abord créer un tableau appelé **posts** (articles) avec les colonnes nécessaires. Vous allez créer un fichier **.sql** qui contient des commandes SQL pour créer le tableau **posts** avec quelques colonnes. Vous utiliserez ensuite ce fichier pour créer la base de données.

- ✓ Ouvrez un fichier appelé **schema.sql** à l'intérieur de votre répertoire **flask_blog**
- ✓ Tapez les commandes SQL suivantes dans le fichier **flask_blog/schema.sql**

```
DROP TABLE IF EXISTS posts;
```

```
CREATE TABLE posts (
```

```
    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    created TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
    title TEXT NOT NULL,
```

```
    content TEXT NOT NULL
```

```
);
```

- ✓ Enregistrez et fermez le fichier.

La première commande SQL est **DROP TABLE IF EXISTS posts**; elle supprime toutes les tables déjà existantes nommées **posts** afin de ne pas créer de confusion. Notez que cela supprimera tout le contenu que vous avez dans la base de données chaque fois que vous utiliserez ces commandes SQL, donc assurez-vous de ne pas écrire de contenu important dans l'application web avant d'avoir terminé ce tutoriel et d'avoir expérimenté le résultat final. Ensuite, **CREATE TABLE posts** est utilisé pour créer la table **posts** avec les colonnes suivantes :

- ✓ **id** : nombre entier qui représente une *clé primaire*, à laquelle la base de données attribuera une valeur unique pour chaque entrée (c'est-à-dire un billet de blog).
- ✓ **created** : l'heure où le billet de blog a été créé. **NOT NULL** signifie que cette colonne ne doit pas être vide et la valeur **DEFAULT** est la valeur **CURRENT_TIMESTAMP**, qui est le moment où l'article a été ajouté à la base de données. Tout comme id, vous n'avez pas besoin de spécifier une valeur pour cette colonne, car elle sera automatiquement remplie.
- ✓ **title** : le titre de l'article.
- ✓ **content** : le contenu de l'article.

Maintenant que vous avez un schéma SQL dans le fichier **schema.sql**, vous l'utiliserez pour créer la base de données à l'aide d'un fichier Python qui générera un fichier de base de données SQLite **.db**. Ouvrez un fichier nommé **init_db.py** dans le répertoire **flask_blog** en utilisant votre éditeur préféré :

- ✓ Ensuite, ajoutez le code suivant.

```
flask_blog/init_db.py
import sqlite3

connection = sqlite3.connect('database.db')
with open('schema.sql') as f:
    connection.executescript(f.read())

cur = connection.cursor()
cur.execute("INSERT INTO posts (title, content) VALUES (?, ?)",
            ('First Post', 'Content for the first post')
            )
cur.execute("INSERT INTO posts (title, content) VALUES (?, ?)",
            ('Second Post', 'Content for the second post')
            )
connection.commit()
connection.close()
```

Vous importez d'abord le module **sqlite3** et ouvrez ensuite une connexion à un fichier de base de données nommé **base de database.db**, qui sera créé une fois que vous aurez lancé le fichier Python. Ensuite, vous utilisez la fonction **open()** pour ouvrir le fichier **schema.sql**. Ensuite, vous exécutez son contenu en utilisant la méthode **executescript()** qui exécute plusieurs instructions SQL en une seule fois, ce qui créera la table **posts**. Vous créez un **objet Cursor** qui vous permet d'utiliser sa méthode **execute()** pour exécuter deux instructions SQL INSERT afin d'ajouter deux articles de blog à votre table **posts**. Enfin, vous validez les modifications et coupez la connexion.

- ✓ Enregistrez et fermez le fichier et exécutez-le dans le terminal en utilisant la commande python :

```
python init_db.py
```

Une fois l'exécution du fichier terminée, un nouveau fichier appelé **database.db** apparaîtra dans votre répertoire **flask_blog**. Cela signifie que vous avez configuré avec succès votre base de données.

Dans l'étape suivante, vous récupérerez les articles de blog que vous avez insérés dans votre base de données et les afficherez sur la page d'accueil de votre application.

2.1.2 Afficher tous les articles

Maintenant que vous avez configuré votre base de données, vous pouvez modifier la fonction d'affichage **index()** pour afficher tous les articles que vous avez dans votre base de données.

- ✓ Ouvrez le fichier **app.py** pour apporter les modifications suivantes :
- ✓ Pour votre première modification, vous importerez le module **sqlite3** en haut du fichier **flask_blog/app.py**

```
import sqlite3  
from flask import Flask, render_template  
...
```

- ✓ Ensuite, vous allez créer une fonction qui crée une connexion à la base de données et la renvoie. Ajoutez-la directement après l'importation :

```
...  
from flask import Flask, render_template  
def get_db_connection():  
    conn = sqlite3.connect('database.db')  
    conn.row_factory = sqlite3.Row  
    return conn  
...
```

Cette fonction **get_db_connection()** ouvre une connexion au fichier de base de données **database.db**, et définit ensuite l'attribut **row_factory** à **sqlite3.Row** afin que vous puissiez avoir un accès par nom aux colonnes. Cela signifie que la connexion à la base de données renverra des lignes qui se comporteront comme des dictionnaires Python normaux. Enfin, la fonction renvoie l'objet de connexion **conn** que vous utiliserez pour accéder à la base de données.

Après avoir défini la fonction **get_db_connection()**, modifier la fonction **index()** pour qu'elle ressemble à ce qui suit :

flask_blog/app.py

```
...  
@app.route('/')  
def index():  
    conn = get_db_connection()  
    posts = conn.execute('SELECT * FROM posts').fetchall()  
    conn.close()  
    return render_template('index.html', posts=posts)
```

Dans cette nouvelle version de la fonction **index()**, vous ouvrez d'abord une connexion à la base de données en utilisant la fonction **get_db_connection()** que vous avez définie précédemment. Ensuite, vous exécutez une requête SQL pour sélectionner toutes les entrées de la table **posts**. Vous implémentez la méthode **fetchall()** pour récupérer toutes les lignes du résultat de la requête, celle-ci renvoie une liste des articles que vous avez insérés dans la base de données à l'étape précédente.

Vous fermez la connexion à la base de données en utilisant la méthode **close()** et vous retournez le résultat du rendu du modèle **index.html**. Vous passez également l'objet **posts** en argument, lequel contient les résultats que vous avez obtenus de la base de données, ce qui vous permettra d'accéder aux articles de blog dans le modèle **index.html**.

Une fois ces modifications apportées, enregistrez et fermez le fichier **app.py**.

Maintenant que vous avez passé les articles que vous avez extraits de la base de données au modèle **index.html**, vous pouvez utiliser une **boucle for** pour afficher chaque article sur votre page d'index.

- ✓ Ouvrez le fichier **index.html** :

✓ Ensuite, modifiez-le pour qu'il ressemble à :

```
{% extends 'base.html' %}

{% block content %}
    <h1>{% block title %} Welcome to FlaskBlog {% endblock %}</h1>
    {% for post in posts %}
        <a href="#">
            <h2>{{ post['title'] }}</h2>
        </a>
        <span class="badge badge-primary">{{ post['created'] }}</span>
        <hr>
    {% endfor %}
{% endblock %}
```

Ici, la syntaxe **{% for post in posts %}** est un Jinja pour boucle, qui est similaire à un Python pour boucle sauf qu'il doit être fermé plus tard avec la syntaxe **{% endfor %}**. Vous utilisez cette syntaxe pour boucler sur chaque élément de la liste des messages qui a été passé par la fonction **index()** dans la ligne **return render_template('index.html', posts=posts)**. Dans cette boucle **for**, vous affichez le titre de l'article dans un titre **<h2>** à l'intérieur d'une balise **<a>** (vous utiliserez plus tard cette balise pour faire un lien vers chaque article individuellement). Vous affichez le titre en utilisant un délimiteur de variable littérale (**{{ ... }}**). N'oubliez pas que **post** sera un objet de type dictionnaire, vous pouvez donc accéder au titre de l'article avec la balise **post['title']**. Vous affichez également la date de création l'article en utilisant la même méthode.

Une fois que vous avez fini d'éditer le fichier, enregistrez-le et fermez-le. Ensuite, naviguez vers la page d'index de votre navigateur. Vous verrez les deux articles que vous avez ajoutés à la base de données sur votre page.

Maintenant que vous avez modifié la fonction d'affichage **index()** pour afficher tous les articles que vous avez dans la base de données sur la page d'accueil de votre application, vous allez afficher chaque article sur une seule page et permettre aux utilisateurs de se relier à chaque article.